

# LLD Practice Platform

Design note: the MVP and its user flow, the domain model behind it, the hybrid evaluation approach that keeps feedback explainable — and the key trade-offs, with direct answers to the five design questions.

## A focused practice loop for Low-Level Design

Choose · Design · Submit · Feedback · History · Try again

# Design Note: LLD Practice Platform

MVP, user flow, domain model, evaluation approach and trade-offs · Bishal Chandra Debnath · September 2026

## 1. The MVP in one paragraph

The product is a focused practice loop for a single learner: choose one of three seeded LLD problems (Parking Lot, Vending Machine, Elevator), design it in a guided structured-text editor — classes with responsibilities and collaborators, relationships with types and justification, and a written rationale — submit, and receive explainable feedback produced by a deterministic rubric engine with optional advisory LLM commentary. Every attempt is kept forever with its report, so history shows progress, and “try again” always means a fresh attempt rather than editing the past. The whole thing is a zero-dependency Node.js monolith: **node server.js** boots the API and the browser UI with no install step, which is the strongest demo posture for a two-day prototype.

| In scope (MVP)                                                                                                                                                                                                                                                                     | Deliberately out (non-goals)                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 seeded problems with requirements, keywords and rubric weights · guided structured-text editor · draft saving · submit + async evaluation · deterministic feedback with evidence · optional LLM advisory layer · attempt history · failure state with retry · 53 automated tests | Authentication & multi-user accounts · real database · microservices, queues, K8s · LMS features (cohorts, assignments, grading) · live collaboration · diagram canvas · admin tooling |

Table 1: MVP boundary — the learner journey is the product; everything else is an extension seam

## 2. User flow

The learner journey is the assignment’s practice loop, made concrete. Each stage maps to exactly one screen and one API resource; the attempt’s status is the single source of truth the UI polls. The loop below is also the failure story: if evaluation is slow the UI keeps polling; if it fails, the attempt becomes FAILED with a reason and a retry action — the learner’s submission is never lost or silently downgraded.

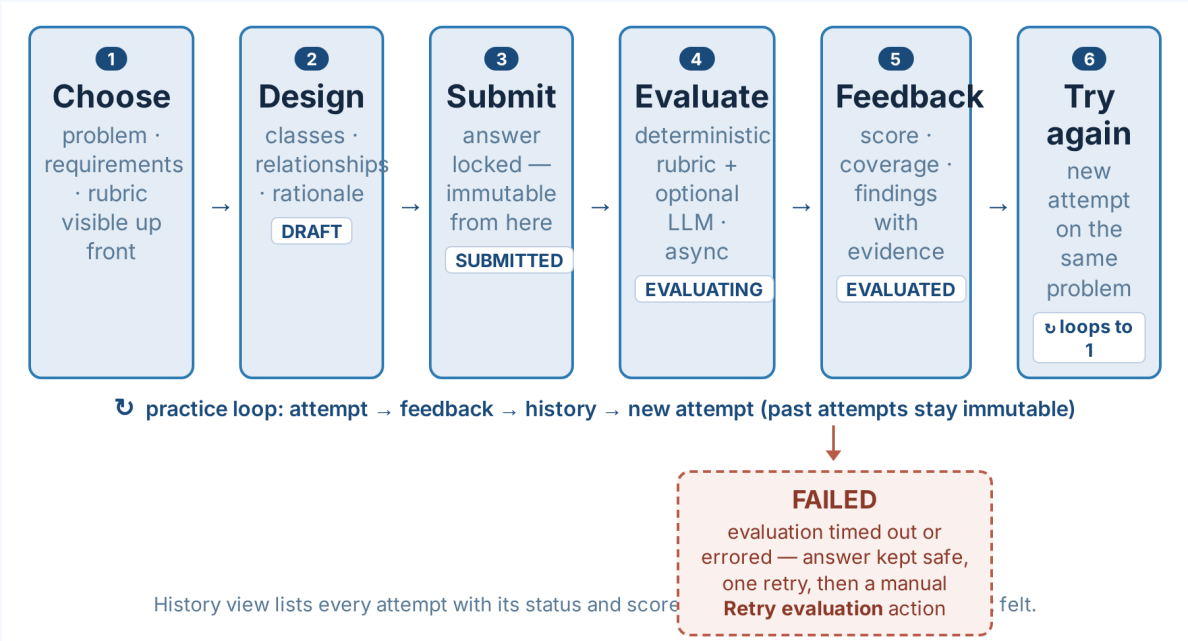


Figure 1: The practice loop and attempt lifecycle (states in caps; the FAILED branch is recoverable)

Two product decisions shape the flow. First, the rubric and requirements are visible *before* the attempt starts — practice becomes a targeted exercise instead of guessing at invisible criteria. Second, submitted answers are

immutable: editing history would destroy its value as evidence of progress, so “try again” always creates a new attempt that will be compared against the old ones.

### 3. Domain model and class architecture

The codebase is organised in five layers with strict top-to-bottom dependencies: the HTTP boundary, the evaluation orchestrator, the evaluation strategy, the domain model, and its repositories. The interesting class is the seam: everything upstream of it speaks the EvaluationEngine interface, so evaluators are swappable strategies rather than hard-wired code paths.

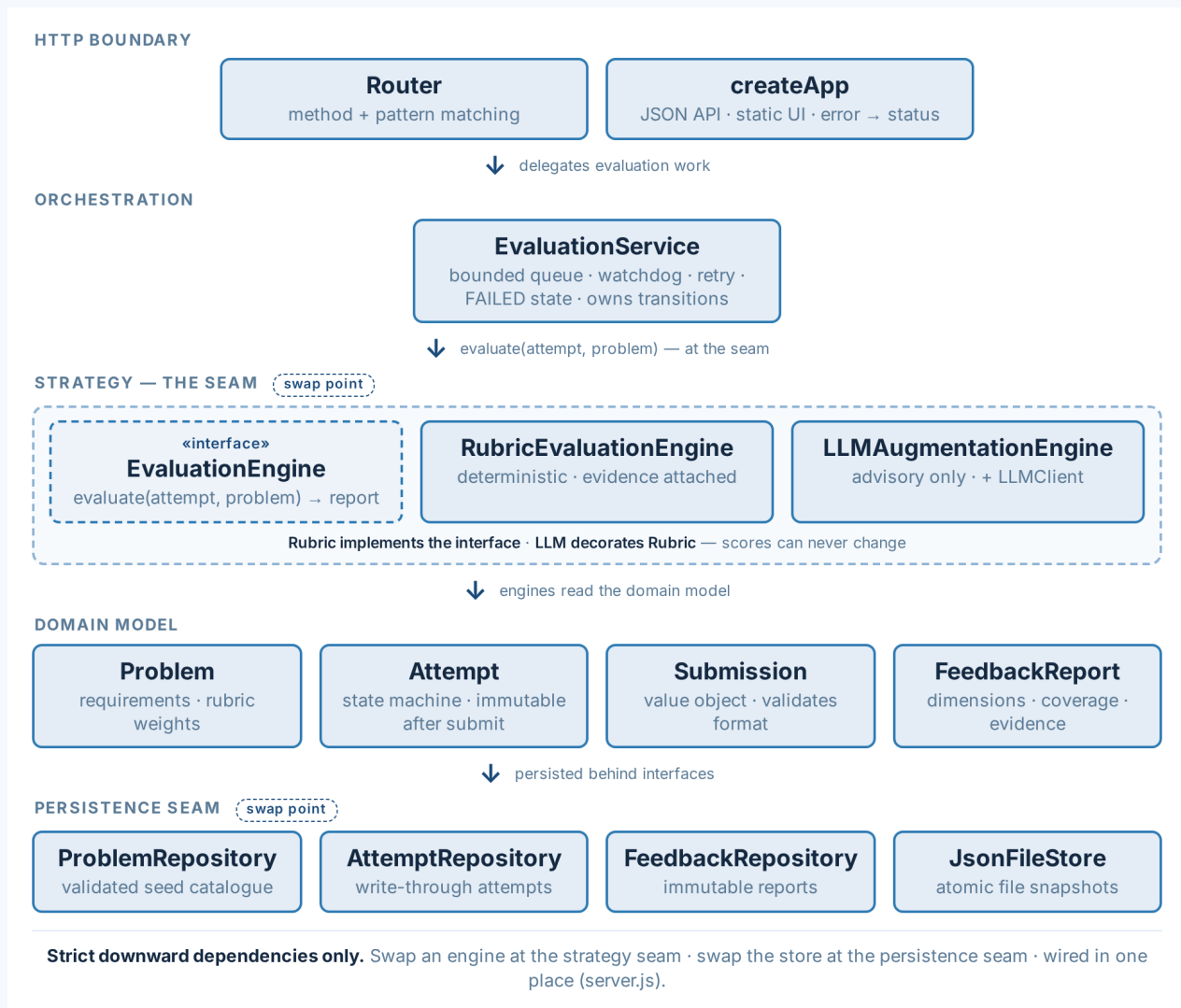


Figure 2: Class architecture — five layers top-to-bottom, the EvaluationEngine seam and the persistence seam

| Class / module    | Responsibility (one line each)                                                                                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Attempt</b>    | Aggregate root. Owns the state machine DRAFT → SUBMITTED → EVALUATING → EVALUATED   FAILED, and enforces every transition — including “no editing after submit” and “FAILED is recoverable”. |
| <b>Submission</b> | Value object. Validates the structured-text format (classes, relationships, rationale) and exposes helpers the evaluators consume (declared names, text corpus).                             |

| Class / module                      | Responsibility (one line each)                                                                                                                                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem / FeedbackReport</b>     | Problem is the curriculum (requirements, keywords, rubric weights — data, not code). FeedbackReport is the immutable evaluation output: dimensions, coverage with evidence, provenance of the LLM layer.                           |
| <b>EvaluationEngine</b>             | The strategy interface: <i>evaluate(attempt, problem) → report</i> . Implementations must not mutate the attempt; the caller enforces the watchdog.                                                                                |
| <b>RubricEvaluationEngine</b>       | Deterministic baseline: requirement coverage by keyword matching with synonym sets, god-class and dangling-reference detection, trade-off vocabulary, ratio-based coverage score. Every finding quotes evidence.                   |
| <b>LLMAugmentationEngine</b>        | Decorator over the deterministic engine. Always runs the baseline first; then best-effort advisory commentary via LLMClient. On timeout, HTTP error or invalid JSON it degrades to the deterministic report — which is never lost. |
| <b>EvaluationService</b>            | The orchestrator: bounded in-process queue, watchdog timeout, one retry, and the FAILED state with a retry path. Owns all status transitions; engines stay dumb and testable.                                                      |
| <b>Repositories + JsonFileStore</b> | In-memory maps with write-through atomic file persistence behind repository interfaces — the swap-for-a-database seam. ProblemRepository validates seeds at boot (broken curriculum stops the server).                             |
| <b>Router + createApp</b>           | HTTP boundary: JSON API, static frontend, path-traversal guard, and a single error-to-status mapping (ValidationError 400, transitions 409, missing 404).                                                                          |

Table 2: Responsibilities of the important classes (the LLD heart of the prototype)

## 4. Evaluation approach

Evaluation is a two-layer pipeline with a strict trust contract. The deterministic layer scores five weighted dimensions — requirement coverage (30%), class design (25%), relationships (20%), rationale & trade-offs (15%), extensibility (10%) — where coverage is computed as a ratio (full 1.0 / partial 0.5 / none 0) and the other dimensions start at 100 and lose points per finding severity (critical 35 / warning 12 / info 4). Findings are concrete and evidenced: a dangling relationship quotes the endpoints, weak coverage quotes the requirement, a god class quotes the responsibility list.

The LLM layer (any OpenAI-compatible endpoint via `LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL`) receives the problem, the submission, and the deterministic findings, and returns JSON with a summary, strengths, improvements and per-dimension notes. It is **advisory only**: the augmentation engine merges the commentary into a separate report section and cannot touch scores — reproducibility and auditability stay intact, and the product still works fully with no API key. When the LLM times out, errors or returns a wrong shape, the report ships deterministic-only with the reason recorded in the report's provenance block, so degradation is visible rather than silent.

Execution is asynchronous by design: submit returns 202 immediately, the attempt moves EVALUATING, and the UI polls. A watchdog bounds each run (default 30s), one retry covers transient failures, and persistent failure lands in FAILED — an infrastructure state with a reason and a retry action, distinct from the learner's answer which is never at fault and never lost.

## 5. Key trade-offs

Six decisions carry the design; each rejected an alternative that a reasonable engineer would defend. The table states the alternatives and the reasoning, in the spirit of “what did you choose not to do, and why”.

| Decision                                 | Alternative rejected                   | Why                                                                                                                                                                                                                  |
|------------------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Deterministic-first, LLM advisory</b> | LLM scores the answer end-to-end       | Same submission must yield the same score; LLM output is unauditible and provider-dependent. Reasoning is where the LLM helps, so that is all it gets.                                                               |
| <b>Structured-text submission</b>        | Free textarea; or parse submitted code | Free text weakens deterministic checks; code-first judges syntax, not design thinking. Structured text keeps every field human-written and parse-friendly.                                                           |
| <b>Immutable attempts</b>                | Editable after feedback                | History must be trustworthy evidence of progress. “Try again” = new attempt, comparable forever.                                                                                                                     |
| <b>Zero-dependency Node monolith</b>     | Next.js + Prisma + UI kit              | Evaluation weights LLD at 25% and implementation at 10%; a framework spends the time budget on tooling and adds an install/build step between clone and demo.                                                        |
| <b>In-process bounded queue</b>          | Job table + background workers + DLQ   | The assignment warns against a distributed-systems project. A cap-2 queue with watchdog and retry answers the real need (never hang, never lose work) at demo scale; the service is the seam for a real queue later. |
| <b>JSON file persistence</b>             | SQLite/Postgres now                    | Write-through atomic snapshots behind repository interfaces are enough for one learner and keep the demo dependency-free; the swap is a repository implementation, not a redesign.                                   |

Table 3: The six load-bearing decisions and their rejected alternatives

## 6. Answers to the five design questions

### Q1. What does a learner need to provide for an attempt to be meaningful?

Three things, all text: the classes (name, responsibilities, collaborators), the relationships (from, to, type, why), and a rationale. That is the minimum a reviewer — human or rule-based — can reason about: responsibilities, structure and intent. Less (a diagram screenshot) is not parseable; more (running code) shifts the exercise from design to syntax. The validator enforces the floor (at least one class, one responsibility each, a 100+ character rationale), and the rubric is visible before the attempt starts.

### Q2. What makes feedback useful when more than one LLD solution is valid?

Feedback that critiques the learner’s own choices instead of diffing against a reference answer. Concretely: every finding is tied to a rubric dimension, quotes evidence from the submission, and names the next action (“split System by role”, “declare Floor as a class”). Scores compare a learner to the rubric, never to another learner. The LLM layer is prompted explicitly that there is no single correct design and that it must reference the learner’s actual classes — and it is advisory, so a quirky model opinion can never distort the reproducible signal.

### Q3. Which parts of evaluation are deterministic, and which benefit from an LLM?

Deterministic: requirement coverage (keyword/synonym matching with plural tolerance), structural checks (class count bands, god classes, dangling references), trade-off vocabulary, extensibility signals, and all scoring. LLM: judging design intent, spotting missing abstractions, and coaching — exactly the reasoning layer rules cannot do. The split is enforced structurally: the decorator can only add a commentary section.

#### Q4. How would the design accommodate another evaluation approach or submission format?

Both are interface seams. A new evaluator (a diagram parser, an AST-based code reviewer, a human-review queue) implements `EvaluationEngine` and is wired in the composition root — nothing else changes. A new submission format implements the same validation contract as `Submission` (the attempt stores an opaque, validated value object), and evaluators declare which formats they can handle. Problems-as-data already carries per-problem rubric weights, so new problems need no code.

#### Q5. What happens if evaluation takes time or fails?

Submit returns 202 immediately; the attempt shows SUBMITTED/EVALUATING while the UI polls. Each run is bounded by a 30s watchdog so nothing hangs forever; one retry covers transient errors; persistent failure lands in FAILED with the reason and a retry action that re-runs the same stored answer. The learner's submission is never lost, and a missing problem or a broken engine is reported as an infrastructure failure, not confused with a bad answer. All of this is in-process and tested — deliberately not a distributed-systems project.

## 7. Testing and reliability

---

The suite (node --test, 53 tests across four files) pins the behaviour the design promises: the attempt state machine including every illegal transition; submission validation edge cases (empty classes, duplicate names, silent truncation regression); the deterministic engine on strong and weak submissions, reproducibility, and evidence-tracing; the LLM decorator in all four failure modes (unconfigured, timeout, HTTP error, invalid JSON) plus the cannot-change-scores contract; the service's happy path, retry, watchdog timeout, unknown-problem and re-evaluation flows, and the concurrency cap; and an end-to-end HTTP journey that boots a real server on an ephemeral port and walks choose → draft → submit → feedback → history, including 400/404/409 paths and the path-traversal guard.

Under load the monolith's honest limits are known and documented: the in-process queue caps concurrency at two, JSON snapshots rewrite whole files, and LLM latency is absorbed by polling rather than sockets. The seams for the real answers — a worker queue behind `EvaluationService`, a database behind the repositories — exist precisely so that scaling is an implementation swap, not a redesign.

A two-minute demo script for a reviewer: run **node server.js**, open **localhost:3000**, pick the Vending Machine problem and skim its visible rubric; start an attempt and fill a small design (one god class does fine — the point is the feedback, not the answer); submit and watch the status move SUBMITTED → EVALUATING → EVALUATED; read the report — coverage per requirement, dimension scores, findings quoting your own words; open My attempts and start a second attempt to see history build; finally run **npm test** for the 53-test suite. Optionally export `LLM_API_KEY` and re-submit to see the advisory coach commentary appear alongside the unchanged deterministic scores.